



Universidad Autónoma de Querétaro
Facultad De Ingeniería



**FACULTAD DE
INGENIERÍA**

Problema del Vendedor Viajero (TSP) con Restricciones

**Algoritmos Metaheurísticos
Tarea 4**

P R E S E N T A

Ing. Daniel Eduardo González Alvarado

Profesor

Dr. Marco Antonio Aceves Fernández

10 de Noviembre de 2025

Índice

1. Introducción	1
1.1. Ciudades	1
1.2. Cálculo de Distancias (Matriz de Haversine)	2
2. Desarrollo	3
2.1. Codificación	3
2.2. Restricciones de Rutas Tabú	4
2.3. Generar Individuo Inicial Valido	5
2.4. Evaluar Aptitud	6
2.5. Generar población	6
2.6. Ordenamiento	7
2.7. Selección	7
2.8. Cruce	8
2.9. Mutación	10
2.10. Ejecucion del Algoritmo	11
2.11. Poblacion Inicial: 100	13
2.11.1. Cruce PMX	13
2.11.2. Cruce PBX	13
2.12. Poblacion Inicial: 500	14
2.12.1. Cruce PMX	14
2.12.2. Cruce PBX	15
2.13. Poblacion Inicial: 1000	16
2.13.1. Cruce PMX	16
2.13.2. Cruce PBX	17
3. Resultados	18
4. Discusión	19
5. Conclusión	19

1. Introducción

El Problema del Vendedor Viajero (Traveling Salesman Problem, TSP) es un caso clásico de optimización combinatoria que busca encontrar la ruta más corta que permite visitar un conjunto de N ciudades exactamente una vez y regresar al punto de origen. A pesar de su formulación simple, el TSP es un problema *NP-hard*, por lo que no existe un algoritmo eficiente capaz de resolverlo de forma exacta en tiempo polinómico para todas las instancias.

Este problema tiene gran relevancia práctica en ámbitos como la logística, la planificación de rutas, la manufactura o la biología computacional. Actualmente, técnicas heurísticas y metaheurísticas como lo son los algoritmos genéticos (GA) se han consolidado como herramientas efectivas para obtener soluciones aproximadas en tiempos razonables, especialmente en escenarios con gran número de ciudades o restricciones complejas.

En este trabajo se utiliza un algoritmo genético para resolver el TSP, aplicando operadores clásicos de cruce y mutación diseñados para preservar la validez de las permutaciones considerando que existen restricciones en las rutas. Entre los operadores de cruce se emplean:

- **PMX (Partially Mapped Crossover)**: mantiene la correspondencia entre segmentos de los padres mediante un mapeo parcial que evita duplicaciones.
- **PBX (Position-Based Crossover)**: conserva ciertas posiciones fijas del primer padre y completa el resto con el orden relativo del segundo.

Para la mutación se implementan:

- **Scramble Mutation**: reordena aleatoriamente un subconjunto de ciudades, modificando el orden local sin alterar la estructura global.
- **Block Transpose Mutation**: intercambia bloques completos dentro de la ruta, explorando nuevas configuraciones manteniendo la coherencia de cada bloque.

Las distancias entre ciudades se calcularon mediante la fórmula del *haversine*, que toma en cuenta la curvatura terrestre para obtener una medida más realista entre coordenadas geográficas.

Finalmente, al extender el TSP hacia versiones con **restricciones en las rutas**, se deben considerar factores adicionales como:

- **Rutas prohibidas o tabúes**: pares de ciudades que no pueden conectarse directamente.
- **Penalizaciones adaptativas**: mecanismos que ajustan la función de aptitud para desalentar soluciones que violen restricciones.

La incorporación de estas restricciones transforma el ejercicio del TSP clásico en una variante más acercada a la realidad, representando escenarios reales para la distribución de mercancías o la planificación de rutas bajo limitaciones operativas.

1.1. Ciudades

Para este ejercicio se utilizaron las siguientes ciudades europeas junto con su etiqueta correspondiente:

- 1 — Berlin
- 2 — Hamburg
- 3 — Stuttgart
- 4 — Munich
- 5 — Paris
- 6 — Lyon
- 7 — Bordeaux
- 8 — Toulouse
- 9 — Amsterdam
- 10 — Brussels
- 11 — Luxembourg
- 12 — Zurich
- 13 — Geneva
- 14 — Vienna
- 15 — Linz
- 16 — Roma
- 17 — Florencia
- 18 — Napoles

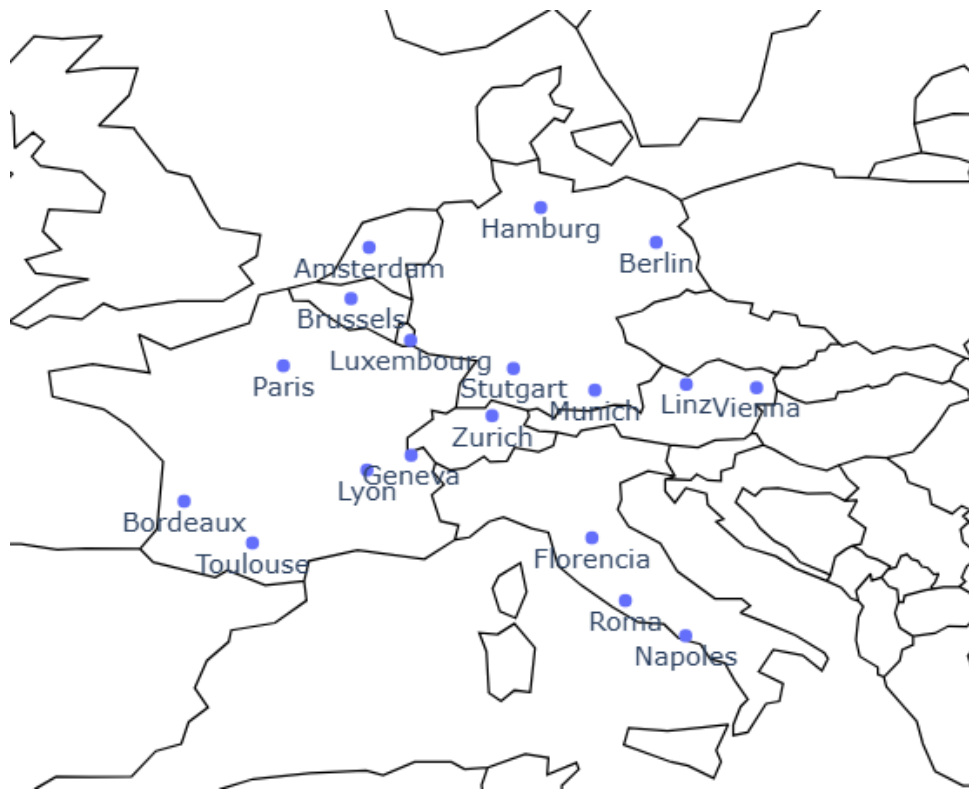


Figura 1: Ciudades consideradas en el TSP.

1.2. Cálculo de Distancias (Matriz de Haversine)

Para calcular las distancias entre las ciudades, se utilizó la fórmula de haversine, que permite determinar la distancia entre dos puntos en la superficie de una esfera a partir de sus coordenadas de latitud y longitud. Esta fórmula es especialmente útil para distancias geográficas, ya que considera la curvatura de la Tierra.

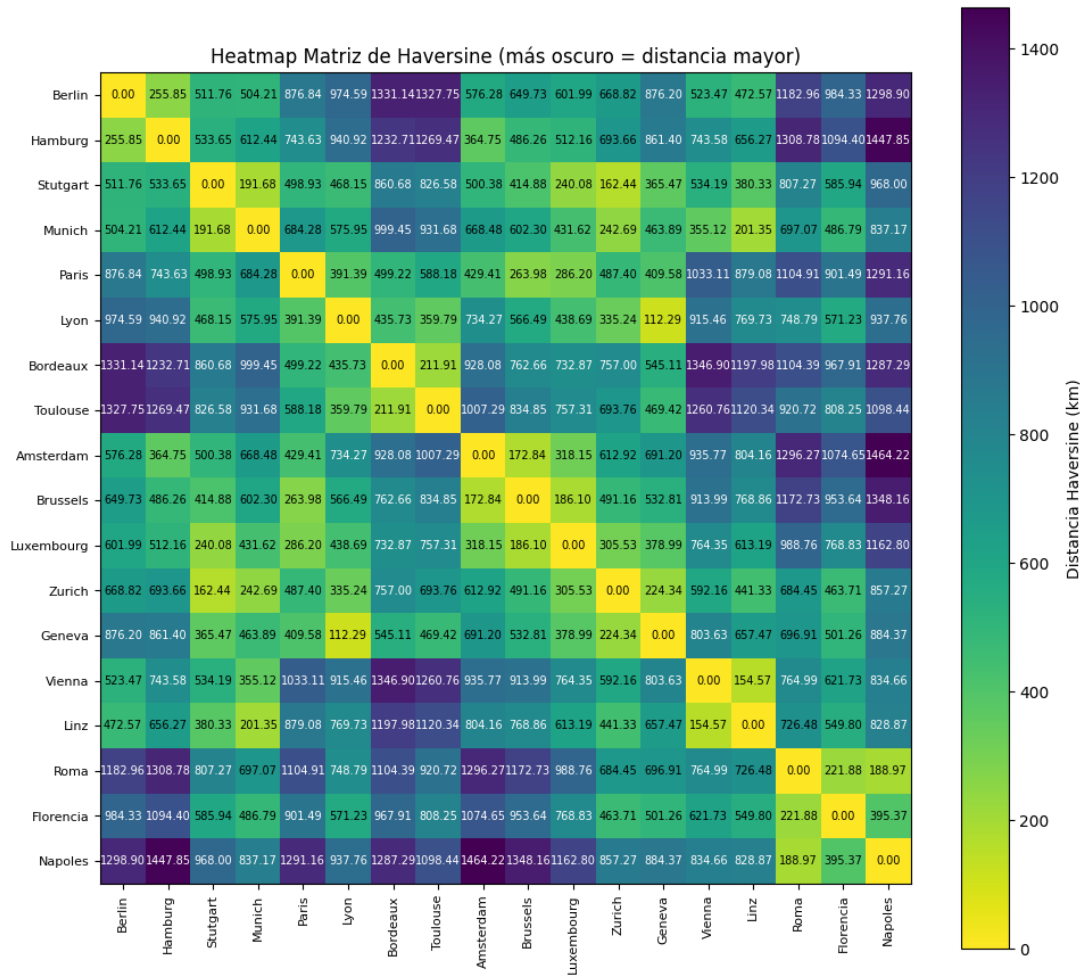


Figura 2: Matriz de distancias entre las ciudades consideradas en el TSP.

Como se puede ver en la figura 2, la matriz muestra las distancias en kilómetros entre cada par de ciudades, en un heatmap donde los colores más oscuros representan distancias mayores y los colores más claros distancias menores. Esto nos proporciona una visión clara de las relaciones espaciales entre las ciudades, lo cual es fundamental para la optimización del TSP.

2. Desarrollo

Partiendo del que el total de combinaciones posibles es $18! = 6,402,373,705,728,000$ sabemos que no es posible evaluar todas las combinaciones sin embargo en este caso debemos tomar en cuenta que existen restricciones en las rutas, por lo que el numero de combinaciones posibles se reduce, pero aun asi sigue siendo un numero muy grande, por lo que se opta por usar un algoritmo genetico.

2.1. Codificación

En este caso, generaremos cada cromosoma como una permutación de las ciudades, representando una ruta posible para el vendedor viajero. Cada gen en el cromosoma corresponde

a una ciudad, y la secuencia de genes determina el orden en que se visitan las ciudades. Sin embargo, debido a las restricciones en las rutas, algunas permutaciones pueden ser inválidas las cuales podemos listar en una “tabla taboo”

[8, 12, 17, 4, 10, 9, 2, 16, 1, 14, 5, 6, 7, 15, 3, 11, 13, 18]

2.2. Restricciones de Rutas Tabú

Las siguientes combinaciones representan pares de ciudades cuya conexión directa está prohibida en el recorrido (rutas tabú):

- | | | |
|--|--|---|
| ▪ (1, 7), (1, 8), (1, 17), (1, 18) | ▪ (7, 1), (7, 2), (7, 4), (7, 14), (7, 18) | ▪ (14, 6), (14, 7), (14, 8) |
| ▪ (2, 6), (2, 7), (2, 8), (2, 17), (2, 18) | ▪ (8, 1), (8, 2), (8, 4), (8, 14), (8, 15), (8, 16), (8, 18) | ▪ (15, 5), (15, 6), (15, 8), (15, 9), (15, 16), (15, 18) |
| ▪ (3, 9), (3, 10), (3, 16) | ▪ (9, 3), (9, 6), (9, 15), (9, 18) | ▪ (16, 3), (16, 8), (16, 11), (16, 15) |
| ▪ (4, 7), (4, 8), (4, 11) | ▪ (10, 3), (10, 17), (10, 18) | ▪ (17, 1), (17, 2), (17, 10), (17, 11) |
| ▪ (5, 15) | ▪ (11, 4), (11, 16), (11, 17), (11, 18) | ▪ (18, 1), (18, 2), (18, 7), (18, 8), (18, 9), (18, 10), (18, 11), (18, 15) |
| ▪ (6, 2), (6, 9), (6, 14), (6, 15) | | |

Cada par (i, j) indica que la conexión entre las ciudades i y j no está permitida dentro del recorrido, y por tanto el algoritmo genético debe evitar dichas transiciones al generar nuevas rutas.

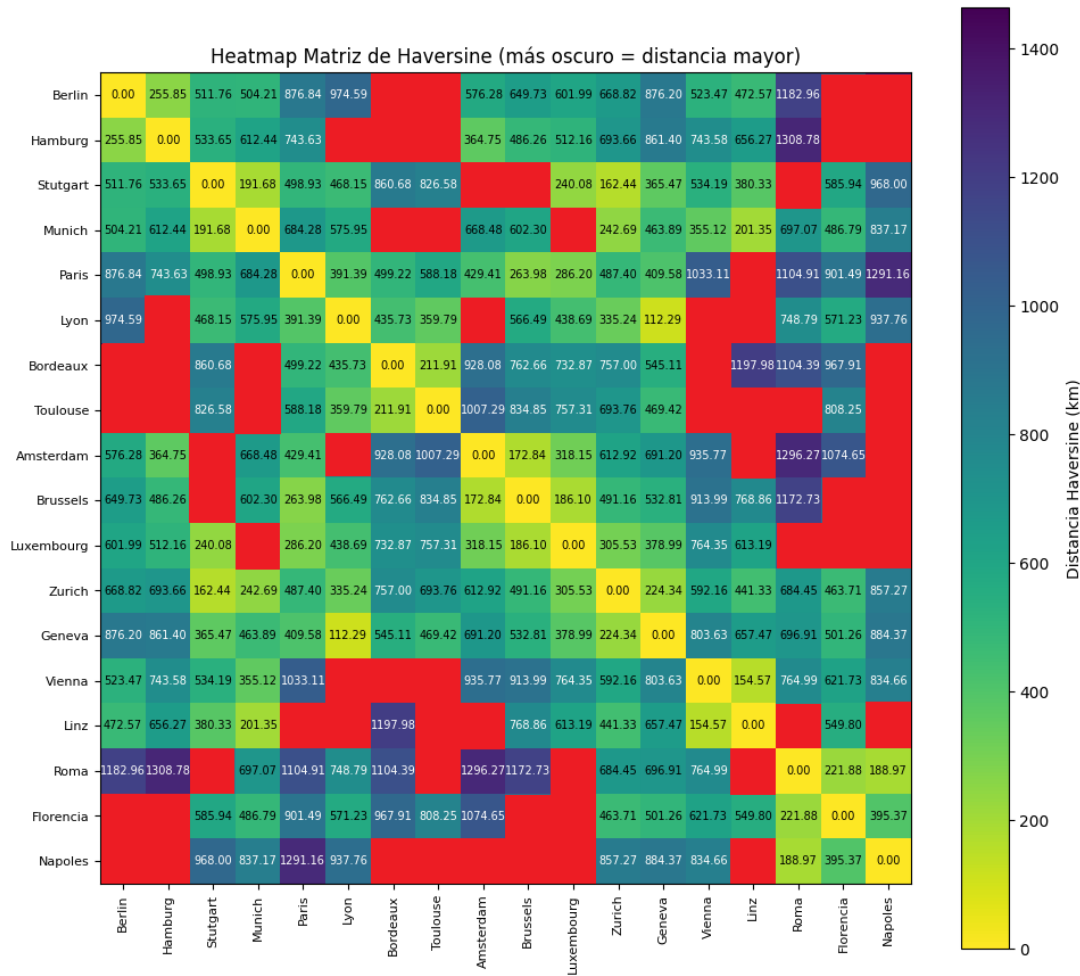
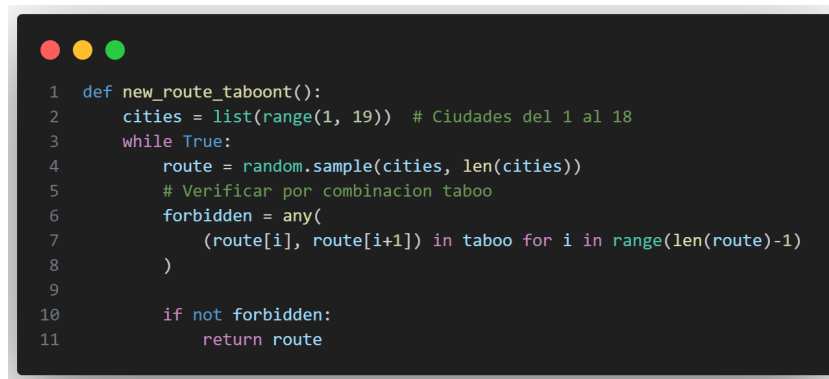


Figura 3: Matriz de Haversine con rutas tabú marcadas en rojo

2.3. Generar Individuo Inicial Valido

Para generar a nuestros individuos iniciales, se crean cromosomas (rutas) de manera aleatoria, asegurando que cada ciudad se visite exactamente una vez en cada ruta. Sin embargo, debido a las restricciones de rutas tabú, es posible que algunas rutas generadas inicialmente sean inválidas. Para manejar esto, se implementa una función que verifica cada par (i, j) en la ruta generada para generar rutas validas.



```

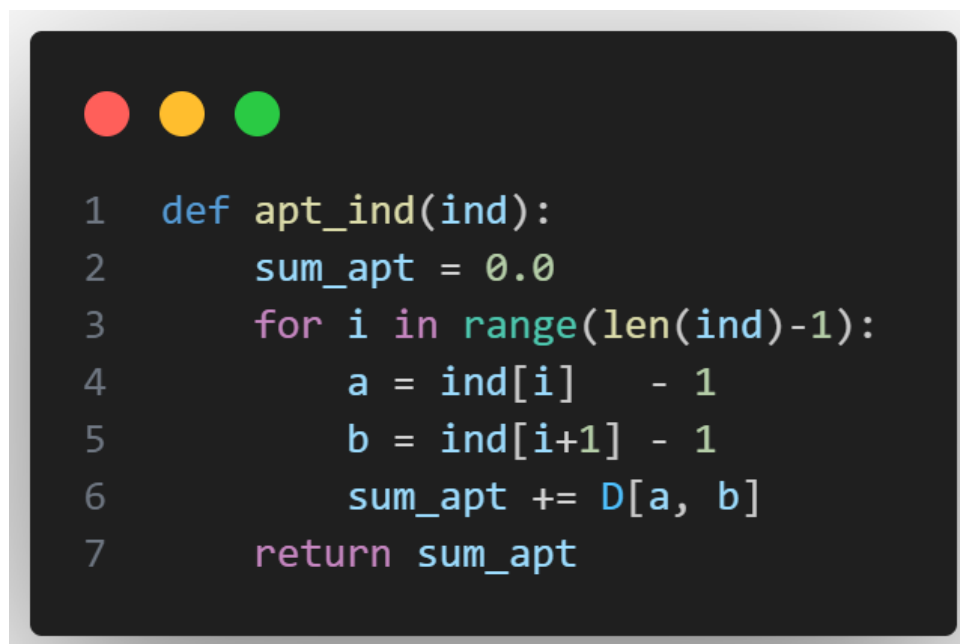
1 def new_route_taboont():
2     cities = list(range(1, 19)) # Ciudades del 1 al 18
3     while True:
4         route = random.sample(cities, len(cities))
5         # Verificar por combinacion taboo
6         forbidden = any(
7             (route[i], route[i+1]) in taboo for i in range(len(route)-1)
8         )
9
10        if not forbidden:
11            return route

```

Figura 4: Funcion generadora de un cromosoma valido

2.4. Evaluar Aptitud

La función de aptitud evalúa la calidad de cada cromosoma (ruta) en la población. En este caso, la aptitud se mide por la distancia total recorrida en la ruta. Esta se calcula sumando las distancias entre ciudades consecutivas en el cromosoma, utilizando la matriz de distancias predefinida, en este caso usaremos la matriz de la figura 3.



```

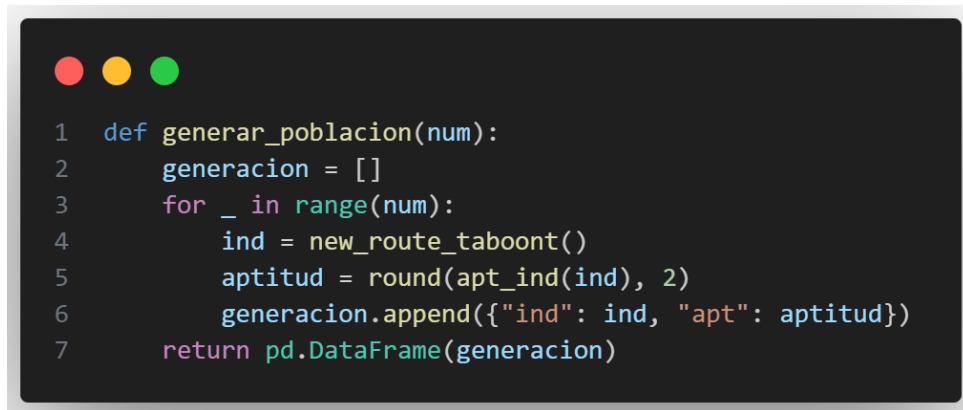
1 def apt_ind(ind):
2     sum_apt = 0.0
3     for i in range(len(ind)-1):
4         a = ind[i] - 1
5         b = ind[i+1] - 1
6         sum_apt += D[a, b]
7     return sum_apt

```

Figura 5: Funcion de aptitud

2.5. Generar población

Una vez definida la función para generar individuos válidos así como para evaluar su aptitud, se crea una población inicial de cromosomas (rutas) utilizando la siguiente función:



```

1  def generar_poblacion(num):
2      generacion = []
3      for _ in range(num):
4          ind = new_route_taboont()
5          aptitud = round(apt_ind(ind), 2)
6          generacion.append({"ind": ind, "apt": aptitud})
7      return pd.DataFrame(generacion)

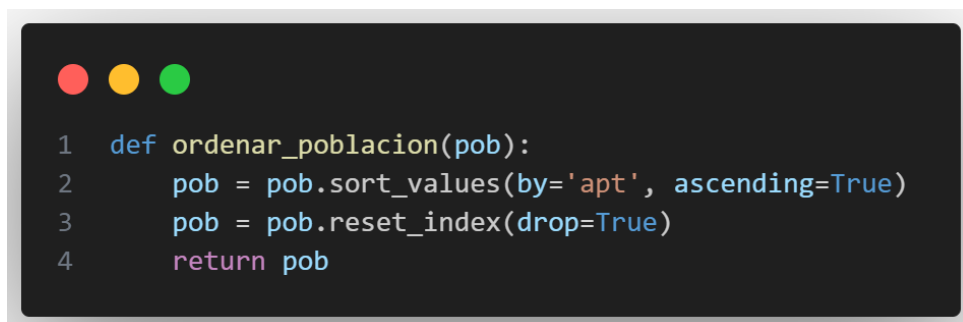
```

Figura 6: Funcion para generar poblacion inicial

Con esta función, generaremos 3 conjuntos de poblaciones iniciales con tamaños de 100, 500 y 1000 individuos cada una para evaluar el desempeño del algoritmo genético bajo diferentes condiciones.

2.6. Ordenamiento

Para seleccionar los mejores individuos de la población, se implementa una función de ordenamiento basada en la aptitud de cada cromosoma.



```

1  def ordenar_poblacion(pob):
2      pob = pob.sort_values(by='apt', ascending=True)
3      pob = pob.reset_index(drop=True)
4      return pob

```

Figura 7: Funcion de ordenamiento de la poblacion

2.7. Selección

Para la selección de padres en el proceso de cruce, usaremos los metodos de torneo y ranking, para lo cual en cada generacion tomaremos la mitad superior de la poblacion ordenada para aplicar rank y la mitad inferior para aplicar torneo, esto con el objetivo de mantener diversidad en la poblacion y evitar caer en un minimo local.

```

1 def parejas_rank(df):
2     df_rs = df.reset_index(drop=True)
3     parejas = []
4     for i in range(0, len(df_rs)-1, 2):
5         if i+1 < len(df_rs):
6             pareja = (df_rs.iloc[i], df_rs.iloc[i+1])
7             parejas.append(pareja)
8     return parejas

```

(a) Funcion ranking

```

1 def parejas_tournament(df):
2     df_rs = df.reset_index(drop=True)
3     parejas = []
4     n = len(df_rs)
5     for i in range(n // 2):
6         pareja = (df_rs.iloc[i], df_rs.iloc[n - 1 - i])
7         parejas.append(pareja)
8     return parejas

```

(b) Funcion torneo

Figura 8: Funciones de seleccion de padres

2.8. Cruce

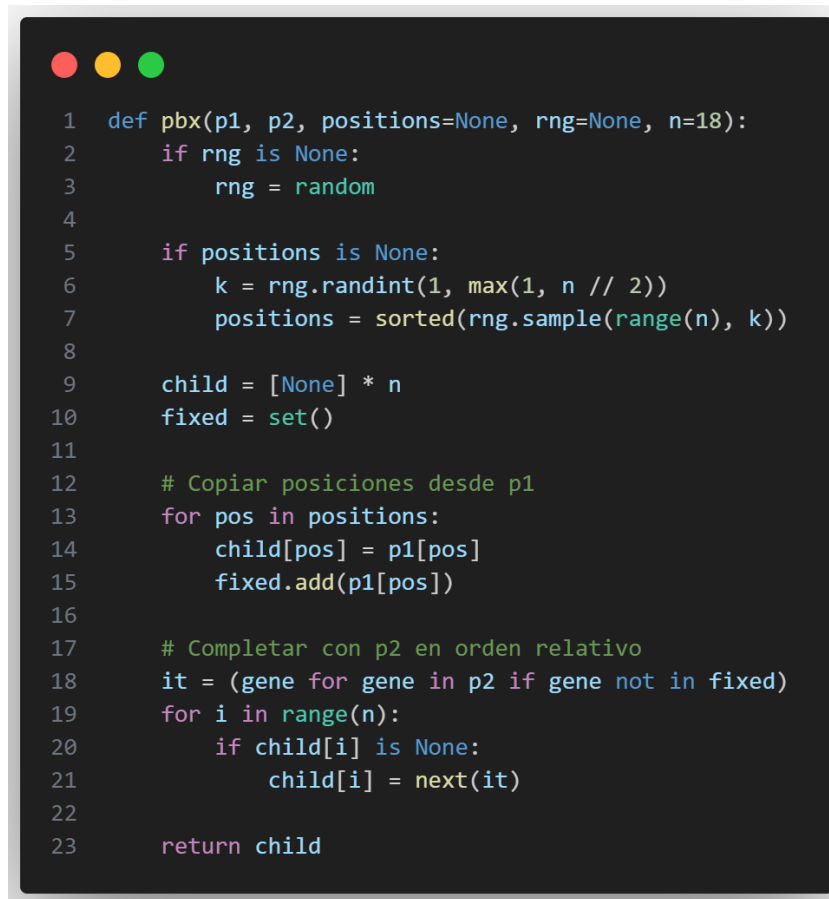
Se emplearán dos operadores de cruce: PMX (Partially Mapped Crossover) y PBX (Position-Based Crossover). El primero selecciona un segmento de la ruta de un padre y lo mapea en la del otro, garantizando que no existan ciudades repetidas mediante un proceso de correspondencia entre genes. El segundo elige posiciones aleatorias del primer padre para copiarlas directamente al descendiente, mientras que las posiciones restantes se completan con las ciudades del segundo padre, respetando su orden relativo.

```

1  def pmx(p1, p2, i=None, j=None):
2      n = len(p1)
3      assert len(p1) == len(p2) and n >= 2
4
5      if i is None or j is None:
6          i, j = sorted(random.sample(range(n), 2))
7
8      h1 = [None] * n
9      h2 = [None] * n
10
11     # Copiar segmento central
12     h1[i:j+1] = p1[i:j+1]
13     h2[i:j+1] = p2[i:j+1]
14
15     # Mapeo bidireccional
16     map12 = {}
17     for a, b in zip(p1[i:j+1], p2[i:j+1]):
18         map12[a] = b
19         map12[b] = a
20
21     seg1 = set(h1[i:j+1]) # genes fijos en h1 (de P1)
22     seg2 = set(h2[i:j+1]) # genes fijos en h2 (de P2)
23
24     # Rellenar h1 con P2 respetando el mapeo
25     for pos in list(range(0, i)) + list(range(j+1, n)):
26         gene = p2[pos]
27         count = 0
28         original_gene = gene
29         while gene in seg1:
30             gene = map12[gene]
31             count += 1
32             if count > n: # Previene ciclo infinito
33                 gene = original_gene
34                 break
35         h1[pos] = gene
36
37     # Rellenar h2 con P1 respetando el mapeo
38     for pos in list(range(0, i)) + list(range(j+1, n)):
39         gene = p1[pos]
40         count = 0
41         original_gene = gene
42         while gene in seg2:
43             gene = map12[gene]
44             count += 1
45             if count > n: # Previene ciclo infinito
46                 gene = original_gene
47                 break
48         h2[pos] = gene
49
50     universo = list(p1)
51     h1 = _repair_no_dups(h1, i, j, universo)
52     h2 = _repair_no_dups(h2, i, j, universo)
53
54     return h1, h2

```

Figura 9: Funcion de cruce PMX



```

1  def pbx(p1, p2, positions=None, rng=None, n=18):
2      if rng is None:
3          rng = random
4
5      if positions is None:
6          k = rng.randint(1, max(1, n // 2))
7          positions = sorted(rng.sample(range(n), k))
8
9      child = [None] * n
10     fixed = set()
11
12     # Copiar posiciones desde p1
13     for pos in positions:
14         child[pos] = p1[pos]
15         fixed.add(p1[pos])
16
17     # Completar con p2 en orden relativo
18     it = (gene for gene in p2 if gene not in fixed)
19     for i in range(n):
20         if child[i] is None:
21             child[i] = next(it)
22
23     return child

```

Figura 10: Funcion de cruce PBX

2.9. Mutación

Se implementarán dos operadores de mutación: Scramble Mutation y Block Transpose Mutation. La primera selecciona un subconjunto aleatorio de ciudades dentro de la ruta y las reordena aleatoriamente, mientras que la segunda intercambia bloques completos de la ruta, explorando nuevas configuraciones manteniendo la coherencia de cada bloque.

```

1 def mut_scramble(ruta, fix_first=True, min_len=2, max_len=None):
2     """
3     Baraja aleatoriamente las ciudades dentro de un subsegmento [i:j].
4     """
5     if len(ruta) < 4:
6         return ruta[:] # nada útil que barajar
7
8     # Soporte ruta cerrada (último = primero)
9     closed = (ruta[0] == ruta[-1])
10    path = ruta[:-1] if closed else ruta[:]
11
12    n = len(path)
13    start_idx = 1 if fix_first else 0
14    usable = n - start_idx
15    if usable < 3:
16        return ruta[:]
17
18    if max_len is None:
19        max_len = usable # sin límite superior
20
21    max_len = max(min_len, min(max_len, usable))
22    l = random.randint(min_len, max_len)
23    i = random.randint(start_idx, n - l)
24    j = i + l
25
26    seg = path[i:j]
27    random.shuffle(seg)
28    new_path = path[:i] + seg + path[j:]
29
30    return new_path + [new_path[0]] if closed else new_path
31

```

Figura 11: Funcion de mutacion scramble

```

1 def mut_block_transpose(ruta, fix_first=True):
2     """
3     Intercambia dos bloques contiguos
4     """
5     if len(ruta) < 4:
6         return ruta[:]
7
8     closed = (ruta[0] == ruta[-1])
9     path = ruta[:-1] if closed else ruta[:]
10
11    n = len(path)
12    start_idx = 1 if fix_first else 0
13    if n - start_idx < 3:
14        return ruta[:]
15
16    # Elegir i, j, k con j-i>0 y k-j>0
17    i = random.randint(start_idx, n - 3)
18    j = random.randint(i + 1, n - 2)
19    k = random.randint(j + 1, n)
20
21    A = path[i:j]
22    B = path[j:k]
23    new_path = path[:i] + B + A + path[k:]
24
25    return new_path + [new_path[0]] if closed else new_path
26

```

Figura 12: Funcion de mutacion block transpose

2.10. Ejecucion del Algoritmo

Finalmente, se integran todos los componentes anteriores en un algoritmo genético completo que itera a través de generaciones, aplicando selección, cruce y mutación para evolucionar la población hacia soluciones óptimas, respetando las restricciones de rutas tabú.

```

1 def Algoritmo_Genetico(poblacion_inicial, num_generaciones, Cruza='PMX', Grafica=False):
2     poblacion = poblacion_inicial.copy()
3
4     df_metricas = pd.DataFrame(columns=["ind", "apt", "Avg_Gen_Fitness", "Delta", "Avg_Delta"])
5     df_metricas = metrics(poblacion, df_metricas, Clean=True)
6
7     patience = 10      # generaciones sin mejora para considerar "estancado"
8     tol = 1e-9         # tolerancia de mejora
9     stagnation = 0
10    best_so_far = float(poblacion.iloc[0]["apt"])
11    mutate_frac = 0.10  # 10% de la población
12
13    mitad1, mitad2 = dividir_mitad(poblacion)
14
15    for gen in range(num_generaciones):
16        parejas_r = parejas_rank(mitad1)
17        parejas_t = parejas_tournament(mitad2)
18
19        if Cruza == 'PBX':
20            hijos_r = cruzar_pbx_parejas(parejas_r)
21            hijos_t = cruzar_pbx_parejas(parejas_t)
22        elif Cruza == 'PMX':
23            hijos_r = cruzar_pmx_parejas(parejas_r)
24            hijos_t = cruzar_pmx_parejas(parejas_t)
25        gen_hijos = pd.DataFrame({'ind': hijos_r + hijos_t})
26
27        if stagnation >= patience:
28            k = max(1, math.ceil(len(gen_hijos) * mutate_frac))
29            idx_to_mutate = random.sample(range(len(gen_hijos)), k=min(k, len(gen_hijos)))
30            # aplicar uno de las dos mutaciones (scramble o block transpose)
31            for idx in idx_to_mutate:
32                ind = gen_hijos.at[idx, 'ind']
33                ind = mut_scramble(ind) if random.random() < 0.5 else mut_block_transpose(ind)
34                gen_hijos.at[idx, 'ind'] = ind
35
36        # evaluar hijos
37        gen_hijos['apt'] = gen_hijos['ind'].apply(apt_ind)
38
39        # integrar con población (como ya lo tenias)
40        poblacion = pd.concat([poblacion, gen_hijos], ignore_index=True)
41        poblacion = ordenar_poblacion(poblacion).iloc[:len(poblacion_inicial)].reset_index(drop=True)
42
43        # actualizar métricas
44        df_metricas = metrics(poblacion, df_metricas)
45
46        curr_best = float(poblacion.iloc[0]['apt'])
47        if (best_so_far - curr_best) > tol: # mejora (menor distancia)
48            best_so_far = curr_best
49            stagnation = 0
50        else:
51            stagnation += 1
52
53        # re-dividir para la próxima generación
54        mitad1, mitad2 = dividir_mitad(poblacion)
55
56        goat_ind = poblacion.iloc[0]['ind']
57        goat_apt = (poblacion.iloc[0]['apt'])/1000
58        print("Mejor ruta: ", goat_ind)
59        print("Distancia en Km: ", round(goat_apt, 3))
60
61        if Grafica:
62            plt.figure(figsize=(10,5))
63            sns.lineplot(data=df_metricas, x=df_metricas.index, y="apt", label="Best Fitness")
64            sns.lineplot(data=df_metricas, x=df_metricas.index, y="Avg_Gen_Fitness", label="Average Fitness")
65            plt.xlabel("Generation")
66            plt.ylabel("Fitness (Distance in meters)")
67            plt.title("Fitness over Generations")
68            plt.legend()
69            plt.show()
70
71    return poblacion, df_metricas, goat_ind
72

```

Figura 13: Funcion principal del algoritmo genetico

Ejecutaremos este algoritmo con las poblaciones iniciales de tamaños 100, 500 y 1000, observando cómo la diversidad y el tamaño de la población afectan la convergencia hacia soluciones óptimas en presencia de restricciones de rutas tabú. Cada ejecución se realizará durante 1000 generaciones y se usaran ambos metodos de cruce (PMX y PBX) para comparar su desempeño en este contexto para esto las poblacion se mantienen constantes con cada metodo.

2.11. Poblacion Inicial: 100

2.11.1. Cruce PMX

Mejor ruta: [11, 3, 12, 4, 15, 14, 1, 2, 9, 10, 5, 7, 8, 6, 13, 17, 16, 18]

Distancia en Km: 4677.348

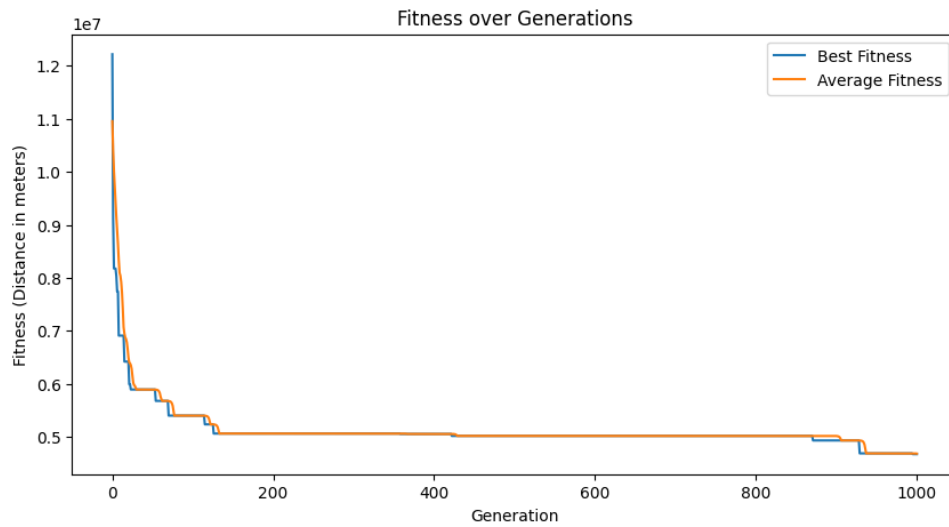


Figura 14: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 100

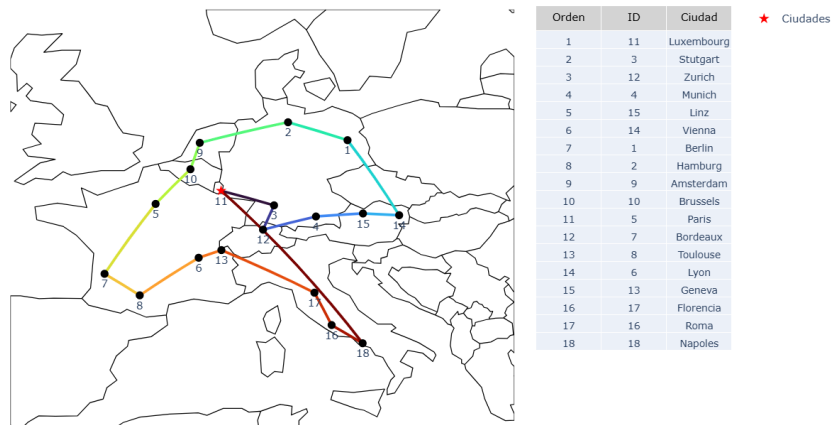


Figura 15: Mejor ruta encontrada con cruce PMX y poblacion inicial 100

2.11.2. Cruce PBX

Mejor ruta: [9, 10, 5, 11, 2, 1, 14, 15, 4, 3, 12, 13, 6, 7, 8, 17, 16, 18]

Distancia en Km: 4927.898

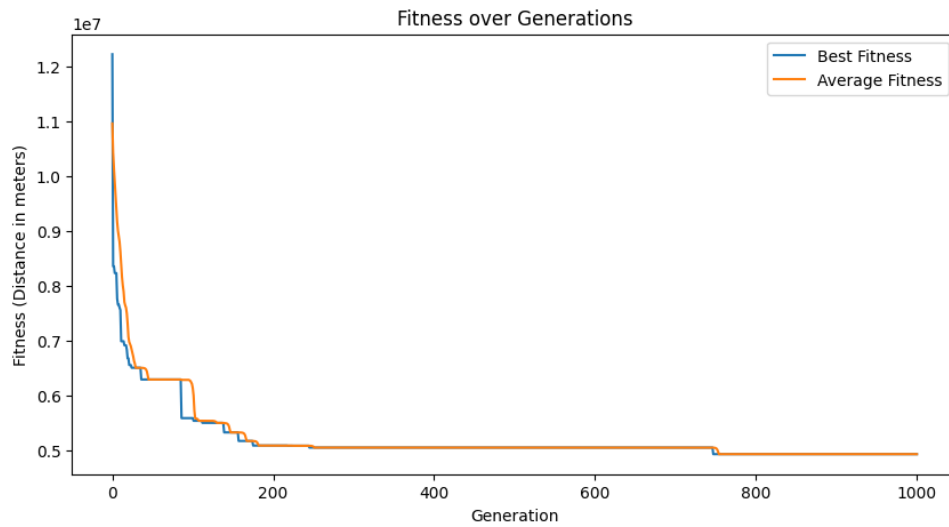


Figura 16: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 100

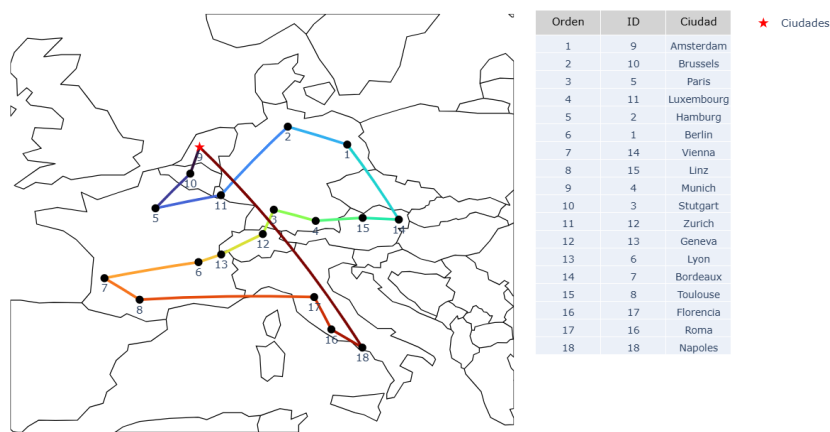


Figura 17: Mejor ruta encontrada con cruce PBX y poblacion inicial 100

2.12. Poblacion Inicial: 500

2.12.1. Cruce PMX

Mejor ruta: [12, 3, 4, 15, 14, 18, 16, 17, 13, 6, 8, 7, 5, 11, 10, 9, 2, 1]

Distancia en Km: 4905.76

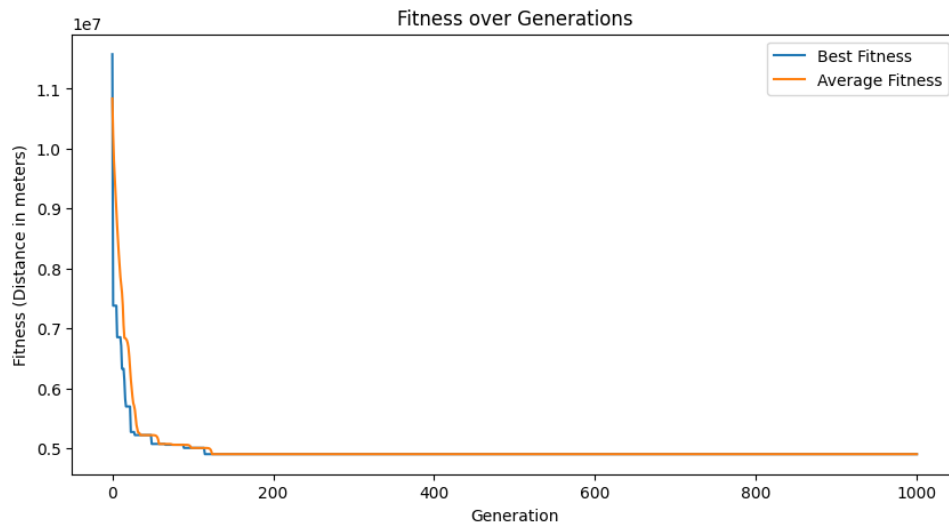


Figura 18: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 500

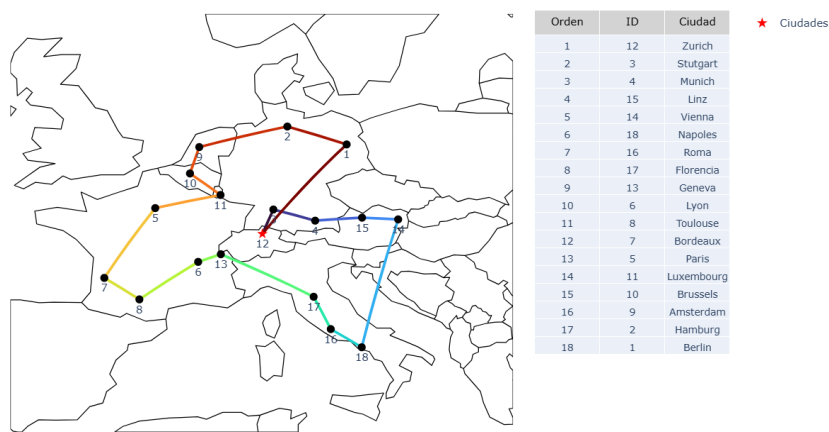


Figura 19: Mejor ruta encontrada con cruce PMX y poblacion inicial 500

2.12.2. Cruce PBX

Mejor ruta: [14, 15, 4, 3, 1, 2, 9, 10, 11, 5, 7, 8, 6, 13, 12, 17, 16, 18]

Distancia en Km: 4607.215

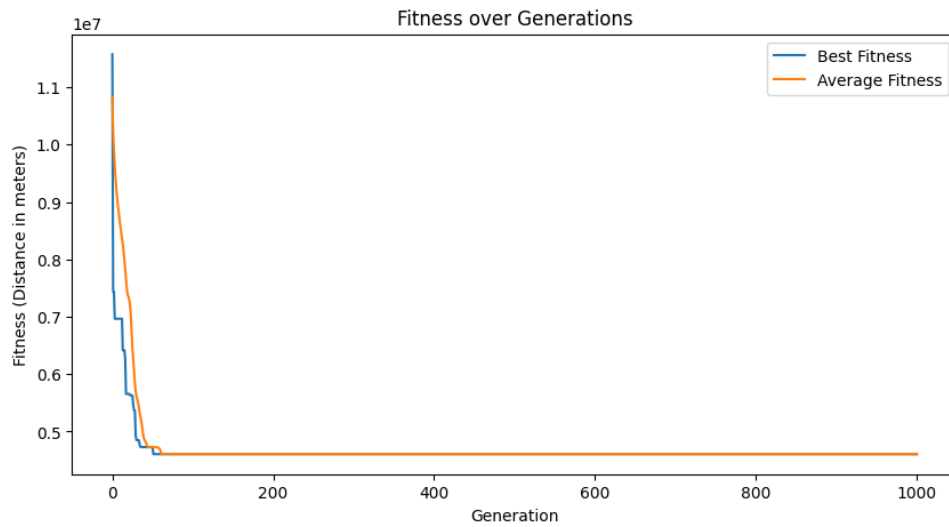


Figura 20: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 500

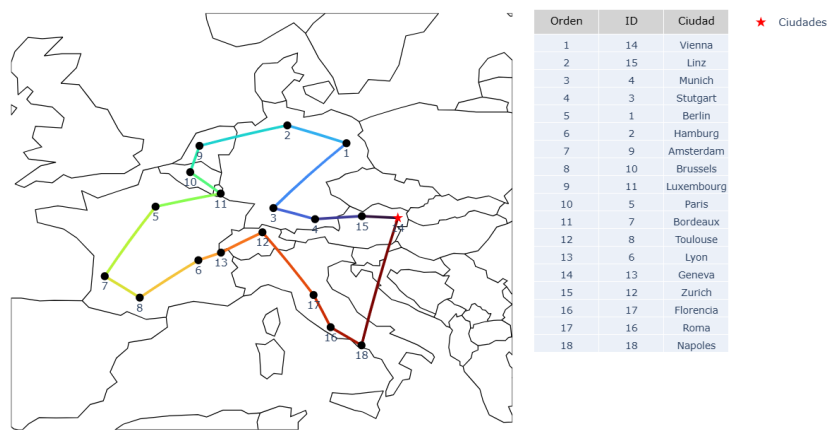


Figura 21: Mejor ruta encontrada con cruce PBX y poblacion inicial 500

2.13. Poblacion Inicial: 1000

2.13.1. Cruce PMX

Mejor ruta: [7, 8, 6, 13, 12, 4, 15, 14, 1, 2, 9, 10, 5, 11, 3, 17, 16, 18]

Distancia en Km: 4610.896

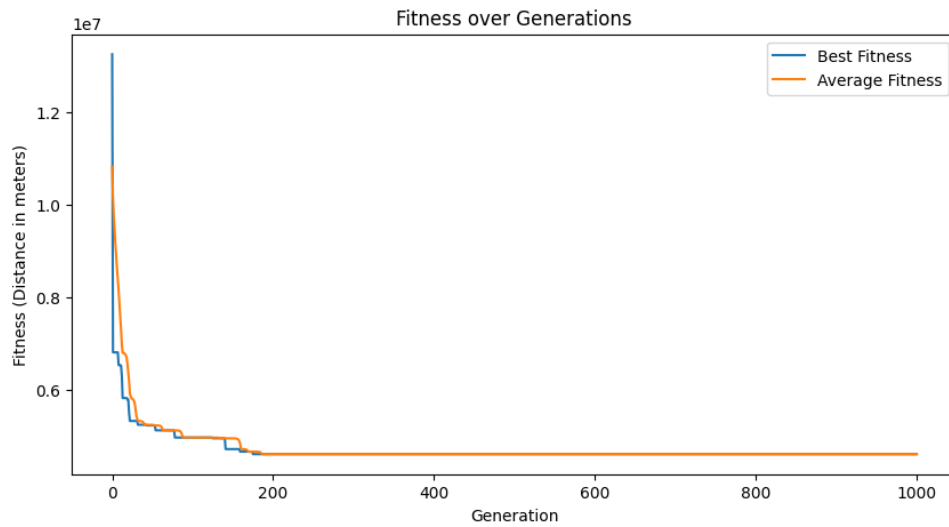


Figura 22: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 1000

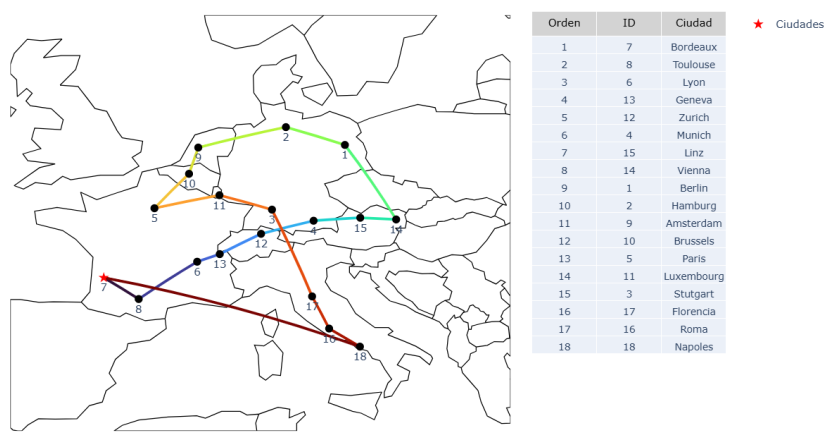


Figura 23: Mejor ruta encontrada con cruce PMX y poblacion inicial 1000

2.13.2. Cruce PBX

Mejor ruta: [1, 2, 9, 10, 11, 5, 7, 8, 6, 13, 12, 3, 4, 15, 14, 17, 16, 18]

Distancia en Km: 4415.911

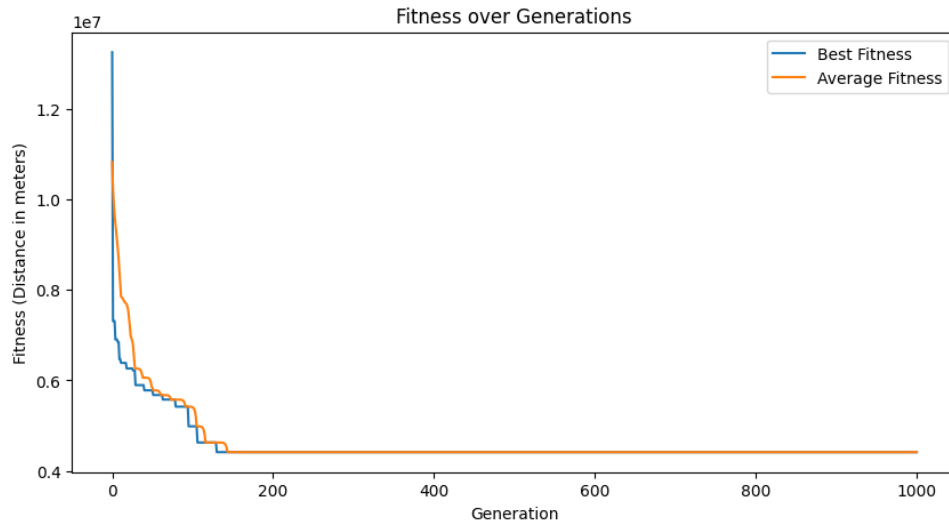


Figura 24: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 1000

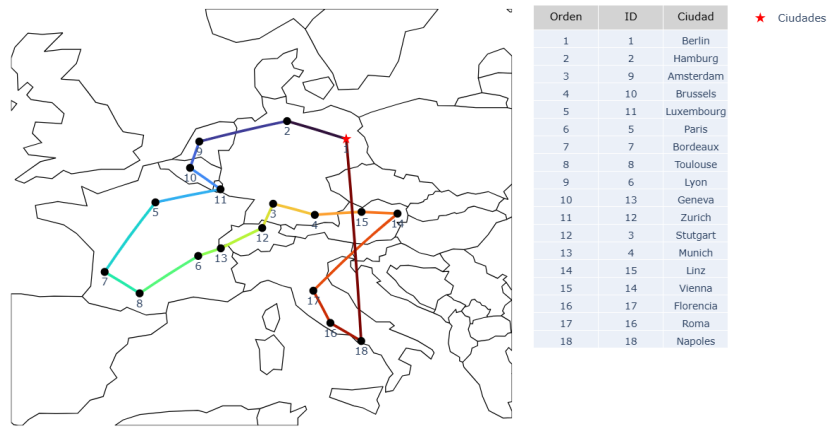


Figura 25: Mejor ruta encontrada con cruce PBX y poblacion inicial 1000

3. Resultados

Podemos observar que con las restricciones de conjuntos (i, j) las rutas se hacen mas complejas de determinar, esto se debe a que ya no es tan facil tener combinaciones validas como aptas, y a su vez esto puede verse en resultados como en la figura 2.13.1 donde existe un “cruce” en la ruta sin embargo esto aunque pareciera extraño el resultado general es bastante bueno al encontrarse entre los mejores obtenidos. Podemos observar nuestros resultados ordenados de la siguiente manera del mejor al peor:

1. 4,415,91Km PBX 1000 – 2.13.2
2. 4,607,21Km PBX 500 – 2.12.2
3. 4,610,89Km PMX 1000 – 2.13.1
4. 4,677,34Km PMX 100 – 2.11.1

5. 4,905,76Km PMX 500 – 2.12.1

6. 4,927,89Km PBX 100 – 2.11.2

Como podemos ver la mejor ruta se encontro con el metodo PBX y poblacion de 1000 esta combinacion se ve favorecida debido al espacion de busqueda inicial ya que tenemos una poblacion muy amplia. Por otro lado la segunda mejor ruta la encontramos con el mismo metodo y una poblacion más pequeña esto demuestra el buen desempeño de este metodo ya que en ambos casos tambien observamos la rapida convergencia del algoritmo en ambos casos siendo alrededor de las 150 generaciones.

4. Discusión

Con los resultados obtenidos podemos determinar que el TSP es una gran ejercicio para probar la calidad de nuestros AG debido a su versatilidad asi como practicidad en casos reales. En este ejercicio podemos ver que estas restricciones nos llevan a soluciones que estan mas fuera de nuestra vision, es decir combinaciones distintas a lo que esperaríamos, ya que en el caso del TSP sencillo, las mejores rutas se encontraban al hacer un circuito, pero en esta ocasion podemos ver que las rutas generan distintas formas por lo que tambien notamos que existen un conjunto variado de combinaciones, no solo un conjunto con las mismas características.

Para nuestra mejor ruta en la seccion 2.13.2 podemos ver que aunque el mapa de la figura 2.13.2 muestra el cierre de la ruta esto solo es una convencion ya que la ruta en si es similar a una “S” llendo de norte a oeste, luego este y por ultimo al sur, esta ruta demuestra estos casos donde la mejor opcion no es un circulo como tal, si no las distancias más cortas.

5. Conclusión

Con base en lo mencionado a lo largo de este reporte asi como en el del TSP simple podemos ver que este ejercicio ofrece un muy buen campo de pruebas para nuestros AG. Para su aplicacion en el mundo real es fundamental tener en cuenta los casos como el que abordamos en este ejercicio donde existen limitantes a las rutas que podemos crear, ya que no siempre nos encontraremos con el camino “ideal” por diversos factores, como clima, caminos en reparacion, rutas cerradas por personas o problemas con el terreno y que aunque no se abordo la situacion tambien podemos encontrar escenarios donde sea obligatorio cumplir con sub-rutas obligatorias, es decir que exista una cadena dentro de nuestro recorrido que tenga que ser visitada en ese orden. Esto vas vez va limitando nuestras opciones asi como aumenta la complejidad del ejercicio, sin embargo nos muestra un panorama más real de lo que podemos ver en el mundo real.

Por otro lado aunque en este caso nos enfocamos en visitar ciudades podemos llevar este mismo ejercicio a otros ambitos, por ejemplo rutas de transporte publico, donde se tome en cuenta todas las estaciones donde se detienen los servicios de transporte, desde autobuses hasta trenes. E incluso podriamos efectuar un AG que nos muestre varias rutas diferentes para cubrir un espacio más amplio. Otro ejemplo de estas aplicaciones podria ser en el diseño de circuitos electronicos donde se busque el mejor diseño para eficientar la manofactura en masa de estas tarjetas, pasando por puntos especificos para los componentes asi como restricciones de que no choquen entre si.

Con esto en mente la aplicación de los AG como herramienta de optimización no se limita solo a los ejercicios comunes que se abordan en clase si no en el mundo real, un caso real de esto lo podemos ver en la cadena McDonald's donde en 1948 los hermanos inventaron el "Speedy Service System" que optimizó el proceso de producción mediante la especialización de tareas, y redujo drásticamente el tiempo de preparación de 30 minutos a solo 30 segundos, entre otras cosas esto se hizo al reorganizar el espacio de la cocina para simular una línea de montaje. Esto lo coloqué rápidamente como un exponente en el mundo de la comida rápida. Partiendo de este caso la misma lógica podríamos aplicarla para tener espacios de trabajo más óptimos en los negocios de servicio.